DEPARTMENT OF COMPUTER APPLICATION

II MCA JAVA PROGRAMMING LAB

THREAD PROGRAMMING

1. Write a program to illustrate creation of threads using Thread extends and runnable interface class. start method start each of the newly created thread. Inside the run method there is display message of your name followed by sleep( ) for suspend the thread for 500 milliseconds).

2. Write a JAVA program that creates threads by extending Thread class. First thread display “Good Morning “ every 1 sec, the second thread displays “Hello “every 2 seconds and the third display “Welcome” every 3 seconds.

Extend this code to use isAlive() and join( ), so that till all treads completes don’t continue main threads to “bye bye see you all”.

3. Write a program to create a class MyThread in this class a constructor, call the base class constructor, using super and starts the thread. The run method of the class starts after this. It can be observed that both main thread and created child thread are executed concurrently.

4. Write a program to get the reference id, name and priority value to the current thread by calling currentThread() method.

5. Write a Java program which first generates a set of random numbers and then determines negative, positive even, positive odd numbers concurrently.

6. Write a Java application create a list of numbers and then sort in ascending order as well as in descending order simultaneously.

7. Write a Java application create a list of numbers and then separate and display odd numbers and even numbers by creating two threads simultaneously.

8. Synchronization in multithreaded programs, and it defines a ThreadSafeCounter class with the necessary synchronization. Is this really important? Can you really get errors by using an unsynchronized counter with multiple threads? Write a program to find out. Use the following unsynchronized counter class, which you can include as a nested class in your program:

```
static class Counter {
   int count;
   void inc() {
      count = count+1;
   }
   int getCount() {
      return count;
   }
}
```

Write a thread class that will repeatedly call the inc( ) method in an object of type Counter. The object should be a shared global variable. Create several threads, start them all, and wait for all the threads to terminate. Print the final value of the counter, and see whether it is correct.

Let the user enter the number of threads and the number of times that each thread will increment the counter. You might need a fairly large number of increments to see an error. And of course there can never be any error if you use just one thread. Your program can use join() to wait for a thread to terminate.

Sample code:

```
//Fruit.java
public class Fruit extends Thread {
private String name;
private int pauseSeconds;
public Fruit (String name, int pauseSeconds) {
this.name = name;
this.pauseSeconds = pauseSeconds;
 }

public void run() {
for (int i = 0; i < 10; i++) {
try {
Thread.sleep(pauseSeconds * 1000); // convert to milliseconds
} catch (InterruptedException e) {
System.err.println(e);
 }
 System.out.printf("%02d ", System.currentTimeMillis( ) / 1000 % 60);
 System.out.println(name + " " + i);
 }
 }
 }

public class Example1 {

public static void main(String[] args) {
Fruit f1 = new Fruit("Banana", 3);
Fruit f2 = new Fruit("Apple", 2);
Fruit f3 = new Fruit("Orange", 4);
f1.start();
f2.start();
f3.start();
}
}
```